

Reviewer Pack — Modular Character Degree Gap in Permanent vs Determinant Cir...

Entry dd4699216f41 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 07:48:45 UTC

Modular Character Degree Gap in Permanent vs Determinant Circuits

Entry ID: dd4699216f41 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 07:48:45 UTC

1. Conjecture

Field A (mathematical branch): Modular Representation Theory **Field B** (complexity object): Monotone Circuit Size for Permanent

Statement:

For a random 3-CNF formula Φ with n variables, let d be its minimal degree of an irreducible representation of the symmetric group S_n over $GF(2)$. Then, the monotone circuit size of the permanent polynomial over $GF(2)$ is at least $\Omega(2^{\{d\}} / n^2)$.

Rationale (proposer’s reasoning):

Modular representation theory of symmetric groups provides invariants that could distinguish algebraic structures of permanent and determinant. The degree of irreducible representations relates to symmetry-breaking patterns, which might encode computational complexity via combinatorial constraints on monotone circuits.

Taxonomy category: GCT (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cad3dec921772440

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.5s

5.1 Generated Python source

```
import random
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_prime(n):
        if n <= 1:
            return False
        for i in range(2, int(n**0.5) + 1):
            if n % i == 0:
                return False
        return True

    def generate_primes_up_to(limit):
        primes = []
        for num in range(2, limit):
            if is_prime(num):
                primes.append(num)
        return primes

    def gcd(a, b):
        while b:
            a, b = b, a % b
        return a

    def lcm(a, b):
        return abs(a * b) // gcd(a, b)

    def generate_3cnf(n, m):
        clauses = []
        for _ in range(m):
            literals = random.sample(range(1, 2*n+1), 3)
            clause = [(-l if random.choice([True, False]) else l) for l in literals]
            clauses.append(clause)
        return clauses

    def young_diagram(n):
        d = 0
```

```

while n > 0:
    d += 1
    n -= d
return d

def permanent(cnf):
    if not cnf:
        return 1
    variables = set()
    for clause in cnf:
        variables.update(abs(lit) for lit in clause)
    n = len(variables)

    def permute(i, j):
        nonlocal cnf
        new_cnf = []
        for clause in cnf:
            if i not in clause and j not in clause:
                new_clause = [lit for lit in clause if lit != -i and lit != -j]
                if new_clause:
                    new_cnf.append(new_clause)
            elif i in clause and j not in clause:
                new_clause = [lit for lit in clause if lit != i and lit != -j]
                if new_clause:
                    new_cnf.append([-lit for lit in new_clause])
            elif i not in clause and j in clause:
                new_clause = [lit for lit in clause if lit != -i and lit != j]
                if new_clause:
                    new_cnf.append([-lit for lit in new_clause])
            else:
                new_clause = [lit for lit in clause if lit != i and lit != j]
                if new_clause:
                    new_cnf.append(new_clause)
        cnf = new_cnf

    def permute_all():
        nonlocal cnf
        for i, j in combinations(range(1, n+1), 2):
            permute(i, j)

    permute_all()
    return len(cnf)

n = 40
m = 3 * n
cnf = generate_3cnf(n, m)
d = young_diagram(n)

circuit_size = permanent(cnf)
metric_value = circuit_size / (2**d / n**2)

conjecture_holds = metric_value >= 1
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "circuit_size_over_bound",

```

```

        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or generate_primes_up_to(30)

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))**0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

re_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'circuit_size_over_bound', 'metric_value': 0.0, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'circuit_size_over_bound', 'metric_value': 28.125, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'circuit_size_over_bound', 'metric_value': 0.0, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'circuit_size_over_bound', 'metric_value': 0.0, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'circuit_size_over_bound', 'metric_value': 9.375, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'circuit_size_over_bound', 'metric_value': 9.375, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'circuit_size_over_bound', 'metric_value': 12.5, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'circuit_size_over_bound', 'metric_value': 0.0, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'circuit_size_over_bound', 'metric_value': 0.0, 'instances_tested': 1, 'conje

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_51da1542.py", line 134, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_51da1542.py", line 134, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    ~~~~~
KeyError: 'seed'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before completing all trials; incomplete results prevent evaluation of support fraction or counterexamples. | next: Re-run test with debug logging to capture full results and ensure 'seed' metadata is properly recorded.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	103795
2	preregistration	ollama_remote	qwen3:8b	0	0	24132
3	novelty	ollama_remote	qwen3:8b	0	0	21087
4	novelty	ollama_remote	qwen3:8b	0	0	15349
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20598
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11751
7	judge	ollama_remote	qwen3:8b	0	0	110015

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 306728 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/dd4699216f41.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/dd4699216f41.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/dd4699216f41.tar.gz (if generated)